

# Exercise Sheet 3

---

## Strong and Weak Ties

5942UE Network Science

Prof. Dr. Michael Granitzer Dr. Kanishka Ghosh Dastidar Dr. Jelena Mitrovic

# 3.A Triadic Closure

---

# Exercise 3.A.1: Open Triads and Predicted Closures

Consider the following friendship network (all edges are **strong** ties unless stated otherwise):

Edges:  $A-B$ ,  $A-C$ ,  $A-D$ ,  $B-C$ ,  $D-E$ ,  $E-F$ ,  $D-F$ .

1. List all **open triads** in the network.
2. For each open triad, which missing edge is predicted to appear by **triadic closure**?
3. Which open triad, if closed, would increase the clustering coefficient of node  $A$  the most?
4. Node  $G$  joins and forms strong ties with  $A$  and  $D$ . Predict which edge forms next.



## Solution:

1. Open triads:  $(B, A, D)$ ,  $(C, A, D)$ ,  $(A, D, E)$ ,  $(A, D, F)$ .

2. Predicted closures:  $B-D$ ,  $C-D$ ,  $A-E$ , and  $A-F$ .

Ranking:  $B-D \approx C-D \approx A-E \approx A-F$ .

3.  **$A$ 's clustering:** Closing  $B-D$  or  $C-D$  increases  $A$ 's clustering coefficient the most:  $C_A$  rises from  $1/3$  to  $2/3$ . Closing  $A-E$  or  $A-F$  leaves  $C_A$  unchanged at  $1/3$  because it adds both one new neighbour and one new edge among  $A$ 's neighbours.

4. **Node  $G$ :**  $G$ ,  $A$ , and  $D$  form a closed triad immediately, since  $A-D$  already exists. The new open triads are  $(G, A, B)$ ,  $(G, A, C)$ ,  $(G, D, E)$ , and  $(G, D, F)$ , so triadic closure predicts one of  $G-B$ ,  $G-C$ ,  $G-E$ , or  $G-F$ . With the given information, these are tied.

# Exercise 3.A.2: Computing Clustering Coefficients

For the graph:  $A-B$ ,  $A-C$ ,  $A-D$ ,  $B-C$ ,  $D-E$ ,  $E-F$ ,  $D-F$ :

1. Compute the **local clustering coefficient**  $C_v$  for every node.
2. Compute the **average clustering coefficient**  $\bar{C}$ .
3. Interpret the result: what does it tell us about the network?
4. Visualise the graph with nodes coloured by  $C_v$ .

## Solution:

### 1. Local coefficients:

- $C_B, C_C, C_E, C_F = 1$  (their neighbours are all connected).
- $C_A = 1/3$  (neighbours  $\{B, C, D\}$ , only  $\{B, C\}$  connected).
- $C_D = 1/3$  (neighbours  $\{A, E, F\}$ , only  $\{E, F\}$  connected).

2. Average:  $\bar{C} = (1 + 1 + 1 + 1 + 1/3 + 1/3)/6 = 14/18 = 7/9 \approx 0.78$ .

3. Interpretation:  $\bar{C} \approx 0.78$  is high, indicating a "cliquey" social structure where friends of friends are likely to be friends.

TODO: check

# 3.B Bridges and Brokers

---

# Exercise 3.B.1: Finding Bridges and Structural Holes

Three cliques:

- $\{1, 2, 3\}$  with edges 1-2, 1-3, 2-3
- $\{4, 5, 6\}$  with edges 4-5, 4-6, 5-6
- $\{7, 8, 9\}$  with edges 7-8, 7-9, 8-9

Inter-clique edges: 3-4 and 6-7.

1. Is 3-4 a **local bridge**?
2. Node 5 in Clique 2 connects to Cliques 1 and 3. Does it sit in a **structural hole**?
3. If node 3 forms a direct edge with node 7, what happens to the hole and the bridges?

## Solution:

- **1. Local Bridge:** Yes. Nodes 3 and 4 have no common neighbours besides each other.
- **2. Structural Hole:** No. Node 5 has indirect access to both outside cliques through nodes 4 and 6, but it is not the main broker. Nodes 4 and 6 sit more directly in the structural holes because they are endpoints of the inter-clique edges.
- **3. Edge 3–7:** The structural hole between Clique 1 and Clique 3 partially closes by creating a direct shortcut. Neither ~~3–4~~ nor ~~6–7~~ is a bridge anymore, because alternative paths now exist through the new cycle.

## Exercise 3.B.2: Identifying Bridges with NetworkX

Build the three-clique graph in NetworkX. Identify bridges and rank edges by edge betweenness.

## Solution:

```
import networkx as nx

G = nx.Graph()
G.add_edges_from([(1,2),(1,3),(2,3),      # Clique 1
                  (4,5),(4,6),(5,6),      # Clique 2
                  (7,8),(7,9),(8,9),      # Clique 3
                  (3,4),(6,7)])           # Bridges

# 1. Bridges
bridges = list(nx.bridges(G))
print("Bridges:", bridges)  # [(3,4), (6,7)]

# 2. Edge betweenness
ebc = nx.edge_betweenness centrality(G)
sorted_ebc = sorted(ebc.items(), key=lambda x: -x[1])
for edge, bc in sorted_ebc[:2]:
    print(f" {edge}: {bc:.4f}")
```

## Interpretation:

- **High Betweenness:** Bridges (3,4) and (6,7) carry all traffic between cliques, giving them maximum edge betweenness.
- **Vulnerability:** Removing these edges fragments the network faster than removing internal clique edges,  demonstrating the importance of "weak" bridging ties for global connectivity.

# 3.C Weak Ties Theorem

---

# Exercise 3.C.1: Verifying the Weak-Ties Theorem

**Weak-Ties Theorem:** Under Strong Triadic Closure (STC), every local bridge is a weak tie.

Check STC and local bridges for:

- **A:** Strong 1-2, 1-3. Weak 2-4. No 2-3.
- **B:** Strong 1-2, 1-3, 2-3. Weak 1-4.
- **C:** Strong 1-2, 2-3, 1-3, 3-4, 2-4.

## Solution:

- **A: STC Violated** (1 tied to 2 and 3, but no 2–3 edge). Theorem premise doesn't hold.
- **B: STC Satisfied**. Edge 1–4 is a local bridge and is **weak**. Theorem holds.
- **C: STC Violated** (2 tied to 1 and 4, but no 1–4 edge). Theorem doesn't apply.

# Exercise 3.C.2: Granovetter's Job Study

1. Why are rarely-seen acquaintances more useful for job finding than close friends?
2. Apply the weak-ties theorem to redundant vs. novel information.
3. Is "unfriending all close friends" good advice based on the theorem?
4. How does LinkedIn fit this framework?

## Solution:

- **1. Novelty:** Close friends often know the same people and information as you. Acquaintances are more likely to connect you to different circles, so they can provide **non-redundant** job leads.
- **2. Structure:** Under STC, local bridges are weak ties. These bridge-like weak ties have low redundancy and are more likely to carry novel information.
- **3. Support vs Reach:** No. Strong ties provide trust, support, and reliable help; weak ties provide reach and novelty. You need both.
- **4. LinkedIn:** LinkedIn helps maintain and search weak or dormant professional ties, making it easier to access information beyond one's close local network.

# 3.D Evidence and Overlap

---

# Exercise 3.D.1: Neighbourhood Overlap Calculation

$$O(u, v) = \frac{|\mathcal{N}(u) \cap \mathcal{N}(v)|}{|\mathcal{N}(u) \cup \mathcal{N}(v)|}$$

(excluding  $u$  and  $v$  from  $\mathcal{N}$ )

For edges:  $A-B$ ,  $A-C$ ,  $A-D$ ,  $B-C$ ,  $B-E$ ,  $C-F$ :

1. Compute  $O(A, B)$ ,  $O(A, C)$ , and  $O(B, E)$ .
2. Rank them. Which is most like a local bridge?

## Solution:

- $O(A, B) = \{C\} / \{C, D, E\} = 1/3$
- $O(A, C) = \{B\} / \{B, D, F\} = 1/3$
- $O(B, E) = \emptyset / \{A, C\} = 0$

**Ranking:**  $O(B, E)$  has zero overlap, making it a perfect **local bridge**. Information entering via  $E$  is likely to be completely novel to the  $A, B, C$  cluster.

# Exercise 3.D.2: Neighbourhood Overlap in NetworkX

Compute average neighborhood overlap within and across factions in the karate club.

## Solution:

```
import networkx as nx
import numpy as np

G = nx.karate_club_graph()

def neighborhood_overlap(G, u, v):
    nu = set(G.neighbors(u)) - {v}
    nv = set(G.neighbors(v)) - {u}
    if not (nu or nv): return 0.0
    return len(nu & nv) / len(nu | nv)

# Compare within-faction vs across-faction
within = [neighborhood_overlap(G, u, v) for u, v in G.edges()
           if G.nodes[u]["club"] == G.nodes[v]["club"]]
across = [neighborhood_overlap(G, u, v) for u, v in G.edges()
           if G.nodes[u]["club"] != G.nodes[v]["club"]]

print(f"Avg Overlap Within: {np.mean(within):.3f}")
print(f"Avg Overlap Across: {np.mean(across):.3f}")
```

## Interpretation:

- **Cross-faction weakness:** Cross-faction edges have significantly lower overlap, confirming they behave as structural bridges between communities.
- **Glue vs redundancy:** Within-faction edges have high overlap, providing structural redundancy. Low-overlap edges are the "glue" holding separate social worlds together.

# Exercise 3.D.3: Practical Lab — Weak Ties in the Karate Club

Use `nx.karate_cclub_graph()` to connect the lecture concepts to measurable graph quantities.

1. Compute the **local clustering coefficient**  $C_v$  for every node.
2. Compute the **neighbourhood overlap**  $O(u, v)$  for every edge.
3. Classify edges as:
  - **weak / bridge-like** if  $O(u, v) < 0.10$
  - **medium** if  $0.10 \leq O(u, v) < 0.30$
  - **strong / embedded** if  $O(u, v) \geq 0.30$
4. Visualise the graph:
  - node colour = clustering coefficient
  - edge colour and width = overlap-based tie strength
5. List the five lowest-overlap edges. Are they mostly within one faction or between factions?

## Solution:

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.karate_club_graph()

def neighbourhood_overlap(G, u, v):
    nu = set(G.neighbors(u)) - {v}
    nv = set(G.neighbors(v)) - {u}
    union = nu | nv
    if not union:
        return 0.0
    return len(nu & nv) / len(union)

# 1. Node-level embeddedness
clustering = nx.clustering(G)

# 2. Edge-level embeddedness
overlap = {
    (u, v): neighbourhood_overlap(G, u, v)
    for u, v in G.edges()
```

## Visualisation:

```
pos = nx.spring_layout(G, seed=7)

edge_colours = []
edge_widths = []
for edge in G.edges():
    o = overlap[edge]
    if o < 0.10:
        edge_colours.append("#d95f02")    # weak / bridge-like
        edge_widths.append(1.0)
    elif o < 0.30:
        edge_colours.append("#7570b3")    # medium
        edge_widths.append(2.0)
    else:
        edge_colours.append("#1b9e77")    # strong / embedded
        edge_widths.append(4.0)

node_colours = [clustering[n] for n in G.nodes()]

plt.figure(figsize=(8, 7))
```

## Interpretation:

- **Low-overlap edges** are the best structural candidates for weak ties. They have few or no shared neighbours, so they connect otherwise separate local neighbourhoods.
- **High-overlap edges** sit inside dense parts of the graph. They are structurally redundant and are reasonable candidates for strong, embedded ties.
- **Faction-crossing edges** usually have lower overlap than within-faction edges, but overlap is a structural proxy, not ground-truth psychology. Some within-faction ties can also be bridge-like if they connect two sparse neighbourhoods.

# 3.E Tie Strength in Practice

---

# Exercise 3.E.1: Tie Strengths in Practice

1. Facebook (symmetric) vs. Twitter (asymmetric): structural implications for overlap?
2. Why do maintained relationships grow slowly while total count grows quickly?
3. How should you design a recommender for **information diversity**?

## Solution:

- **1. Symmetry:** Facebook ties are reciprocal, leading to higher local clustering and overlap. Twitter links are closer to structural bridges with near-zero overlap.
- **2. Cognitive limits:** Maintained ties are bounded by the Dunbar number and social attention. Passive links are structural weak ties with high novelty but low investment.
- **3. Diversity:** Recommend content from **low-overlap connections**. These structural bridges provide access to disjoint social bubbles and non-redundant information.

## Exercise 3.E.2: Simulating Tie Strength and Overlap

Assign synthetic strength values to karate-club edges (within-faction strong, across-faction weak) and compute the correlation with neighborhood overlap.

## Solution:

```
import networkx as nx
import numpy as np

G = nx.karate_club_graph()

# Assign synthetic strength: within-faction = strong (0.8), cross = weak (0.2)
strengths, overlaps = [], []
for u, v in G.edges():
    s = 0.8 if G.nodes[u]["club"] == G.nodes[v]["club"] else 0.2
    strengths.append(s)
    overlaps.append(neighborhood_overlap(G, u, v)) # defined in 3.D.2

# Correlation
r = np.corrcoef(strengths, overlaps)[0, 1]
print(f"Pearson correlation: {r:.3f}") # Typically > 0.6
```

## Interpretation:

- **Empirical support:** The strong positive correlation confirms that tie strength and neighborhood overlap are linked: close-knit groups share friends, while bridging ties remain socially isolated.
- **Structural exceptions:** High-strength ties with low overlap may indicate bridging relationships that involve significant social effort (e.g., cross-community collaborations).